

IIoT Communication Protocols: A Comparative Analysis of MQTT, OPC UA, and CoAP for Industrial Networks

1st Ikramul Hassan Sazid

Electronic Engineering (B.Eng.)

Hamm-Lippstadt University of Applied Sciences

Lippstadt, Germany

ikramul-hassan.sazid@stud.hshl.de

2nd Muhammad Aamir Ejaz Ejaz

Electronic Engineering (B.Eng.)

Hamm-Lippstadt University of Applied Sciences

Lippstadt, Germany

muhammad-aamir.ejaz-ejaz@stud.hshl.de

3rd Md Ruhul Kuddus Saleh

Electronic Engineering (B.Eng.)

Hamm-Lippstadt University of Applied Sciences

Lippstadt, Germany

md-ruhl-kuddus.saleh@stud.hshl.de

Abstract—The Industrial Internet of Things plays an important part in the digital transformation of contemporary industry. The application of Internet of Things technology in industrial settings allows for the collection, exchange, and analysis of data in order to increase efficiency, flexibility, and minimize costs. The development of Industry 4.0 has created a need for effective communication protocols, which include MQTT, CoAP, and OPC UA. While this paper theoretically discusses all three standards, it conducts a practical comparative analysis strictly between MQTT and OPC UA. These two protocols are evaluated within a simulated industrial environment to compare their latency, message overhead, architectural design, and security performance. Furthermore, the seminar paper also explores the aspects of data acquisition, monitoring, cloud connectivity, as well as security and reliability in relation to the IIoT system. By carrying out this research, it is possible to demonstrate the relevance of appropriate communication methods for intelligent industrial systems.

I. INTRODUCTION

Industry 4.0 represents a new phase of industrial development where the Industrial Internet of Things (IIoT) plays an important role. In this environment, devices, systems, and applications communicate with each other to exchange data and support intelligent industrial operations. IIoT systems work in harsh environments such as high temperature, vibration, and difficult industrial conditions. These devices are designed for a long lifespan, usually more than 15 years. The main purpose of IIoT is to prevent process interruption and maintain safety, therefore it gives higher priority to availability, integrity, and confidentiality [1]. In case of threats, the system must continue operation instead of shutting down for repair. Because of this, failure in IIoT devices can cause process interruption, production loss, and possible physical dangers [1].

With the rise of Industry 4.0, the manufacturing sector is undergoing a major digital transformation [2], with growing emphasis on the data environment within production facilities

[3]. To generate valuable insights from machinery and improve operational decision-making, seamless data exchange is essential [2], [3]. This data must be securely transmitted through reliable industrial communication protocols and structured into a higher-level ontology [3]. As a result, the shift toward digitalisation has increased the importance of selecting and implementing appropriate communication standards [2]. Since numerous standards have been developed to manage these complex data environments, evaluating the specific protocols used for communication has become a crucial next step [2], [3]. Therefore, this research presents a comparative analysis of three widely used IIoT communication protocols: MQTT, CoAP, and OPC UA. In this study, these protocols are evaluated based on latency, message overhead, and security performance. By completing this analysis, we will see their strengths and suitability for modern intelligent industrial systems.

II. OVERVIEW OF IIoT COMMUNICATION REQUIREMENTS

A. Why Industries Require Specialized Communication

Industrial networks need to ensure strict real-time performance and high reliability while handling different types of traffic that vary in importance. This includes control loops, sensor data, and configuration information. Unlike regular Internet protocols that try their best, industrial communication standards are specifically created to guarantee reliable and consistent data sharing between different devices and manufacturers, even when things go wrong. [5].

B. Technical Requirements

1) *Real-Time Communication*: The ability of a system to transmit and process data within a precisely defined and predictable time frame is referred to as **real-time** performance. This is crucial for applications where time and synchronisation

are crucial, like robotics, motion control, and process automation. Industrial systems are classified into:

- **Hard Real-Time:** Missing a deadline leads to catastrophic failure (e.g., medical devices, nuclear plant control systems). [5]
- **Soft Real-Time:** Missed deadlines degrade performance but do not cause total failure (e.g., monitoring dashboards, video surveillance). [5]

Fig.1. shows how the usefulness changes over time for hard and soft real-time systems when a deadline is not met.

2) *Low Latency:* To guarantee deterministic communication, prevent data loss, and maintain the synchronisation essential for real-time process control and automation, IIoT applications must have ultra-low latency, which lies between 1ms and 10 ms. This is necessary for time-sensitive industrial activities. [6].

3) *Security:* IIoT, or Industrial Internet of Things, greatly expands the number of potential security risks. This happens because new devices and communication methods are added, which can make systems easier to interfere with. As a result, there can be problems with controlling processes, stealing sensitive ideas, losing important company information, and even spying on businesses. Even though the Industrial Internet of Things (IIoT) greatly affects many industries, a big obstacle to using IIoT is the problems related to cybersecurity. These issues make it very hard to take full advantage of what IIoT can offer [7]

4) *Scalability:* The rapid increase in connected IoT devices has put a lot of strain on communication systems, leading to new needs for connectivity, security, and the ability to scale up effectively. Modern systems need to allow for horizontal growth, which means adding more devices without harming the performance of the network. [8].

5) *Interoperability:* The RAMI 4.0 reference model says that industrial systems need to work together completely. This means that both physical and virtual parts should be able to talk to each other easily at every level of the network, no matter who made them or how they are organized. This is important

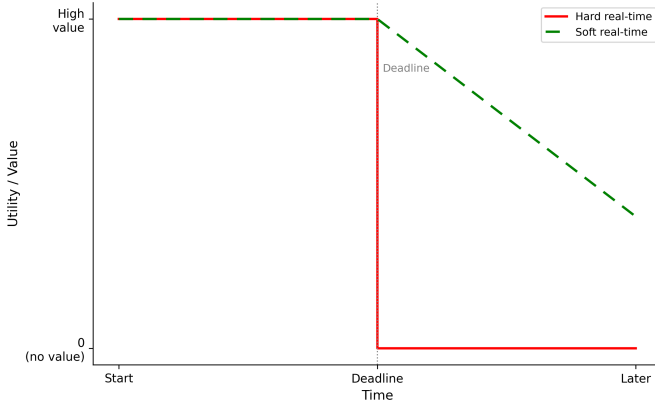


Fig. 1. Hard and Soft Real-Time

for getting different vendors to work together in IIoT settings [8].

C. Operational Requirements: RAMS(S)

Beyond technical performance, IIoT systems must also satisfy operational dependability requirements, collectively described by the RAMS framework [9] which is shown in the **TABLE I**

TABLE I
RAMS(S) FRAMEWORK FOR IIOT DEPENDABILITY

Letter	Property	Description
R	Reliability	System performs required function without failure over specified time
A	Availability	System is accessible and operational when required
M	Maintainability	System can be restored to operational state quickly after failure
S	Safety	No harm to people, equipment, or environment on failure
(S)	Security	Protection against unauthorized access and cyber threats

The additional Security component (S) was incorporated into the traditional RAMS framework as IIoT systems became internet-connected, introducing new cybersecurity attack surfaces that were not present in isolated legacy industrial networks [10].

III. CORE IIoT COMMUNICATION PROTOCOLS

Selecting the suitable application-layer communication protocol is important for Industrial Internet of Things (IIoT) systems because every industrial environment has different operational requirements based on speed, bandwidth consumption, and scalability [13]. In this research MQTT, CoAP, and OPC UA protocols are selected because they represent different modern IIoT communication requirements.

Some popular protocols like HTTP and AMQP are not considered in this work because they require higher bandwidth, create more latency, and increase architectural complexity in constrained industrial systems [13]. Instead, MQTT and CoAP are selected due to their lightweight and scalable design which provides efficient and reliable communication in resource-limited and high-latency IoT networks [4], [13]. Finally, OPC UA is selected to support complex enterprise integrations while providing platform independence and robust security features for maintaining industrial data integrity [13], [14].

A. MQTT (Message Queuing Telemetry Transport)

1) *Background and History:* MQTT was originally developed in 1999 by Andy Stanford-Clark (IBM) and Arlen Nipper for monitoring oil pipelines via satellite links with limited bandwidth. It was sent to OASIS in 2013 and approved as an OASIS Standard in 2014. Since then become one of the most widely adopted protocols for IIoT deployments. The latest version, MQTT 5.0, was released in 2019, introducing enhanced error reporting, session management, and security features [11].

2) *Architecture and Communication Model*: MQTT operates on a publish-subscribe model consisting of three core components [11]:

- **Publisher**: A publisher is any device or software that creates messages and sends them to the messaging system through a specific topic .
- **Broker**: A central server (e.g., Mosquitto, HiveMQ) that receives published messages and distributes them to all interested subscribers.
- **Subscriber**: A subscriber is a device or app that shows interest in a certain topic and gets all the messages that are sent out about that topic through the broker.

This decoupled architecture means publishers and subscribers do not need to know each other's identity or be simultaneously online, which makes MQTT highly suitable for unreliable network conditions.

3) *Technical Specifications*: MQTT operates over TCP/IP on port 1883 (unencrypted) and port 8883 (TLS-encrypted). The protocol header is only 2 bytes, making it one of the most lightweight messaging protocols available. Three Quality of Service (QoS) levels are supported [11], [12]:

- **QoS 0** (At most once): Fire-and-forget delivery; no acknowledgement .
- **QoS 1** (At least once): Guaranteed delivery, but duplicates possible .
- **QoS 2** (Exactly once): Guaranteed single delivery; highest overhead .

4) *Key Features*:

- **Last Will and Testament (LWT)**: Allows the broker to notify subscribers if a client disconnects unexpectedly.
- **Retained Messages**: The broker stores the last message on a topic so new subscribers receive it immediately upon connection.
- **Keep-Alive Mechanism**: Periodic PING messages between client and broker ensure connection health.

B. OPC UA

Open Platform Communications Unified Architecture (OPC UA) is a communication standard that enables secure, reliable and interoperable communication between industrial devices. It was released in 2006 as an improvement of the traditional OPC standard. OPC UA provides a platform-independent method for accessing process variables from industrial devices produced by different vendors. Due to its capability to connect hardware devices and software systems such as SCADA and MES, OPC UA is widely used in Industrial Internet of Things (IIoT) environments. IT harmonize communication between different industrial automation networks and provides a common platform for industrial data exchange [15].

1) *System Architecture and Data Modeling*: OPC UA provides more than a simple communication mechanism as it also offers a structured framework for data representation and transmission [15]. The architecture mainly works on a secure client-server model based on service-oriented communication, but it also supports a publish/subscribe model where a central

key server is used to distribute symmetric keys for secure communication between devices. [15], [16]

To improve data transmission performance, OPC UA defines a binary mechanism based on TCP protocol which provides efficient communication in intranet environments [15]. For external networks, OPC UA also supports communication through web services and HTTP/XML mechanisms which allow access through firewalls [15]. Industrial systems have different requirements compared to traditional IoT networks because they often require real-time communication [15]. To support these requirements, industrial fieldbus technologies have evolved towards Real Time Ethernet (RTE) solutions such as Profinet and EtherCAT [15]. Here we can see that OPC UA can operate together with these communication channels and automatically connect devices to RTE networks after device identification, ensuring continuous exchange of time-critical information [15].

Another important feature of OPC UA is its data modeling capability [15]. It provides a common framework for developing structured information models and hierarchical representation of complex process data [15]. This information is stored within the OPC UA server address space and can be accessed systematically by client applications [15]. In this process we will see how OPC UA allows efficient organization and management of industrial data.

2) *Deployment in Industrial Environments and Auto-Configuration*: In traditional industrial environments, the integration of Programmable Logic Controllers (PLCs) with OPC UA servers requires significant manual configuration. Process variables must be mapped manually into the server information model, and any device replacement or system failure often results in production downtime and additional configuration efforts. [15]

To overcome these challenges, modern industrial systems are moving towards Plug and Produce (PnP) auto-configuration approaches. In this method, the OPC UA server can automatically manage connected devices through a basic TCP/IP network. The process generally consists of three stages. [15]

Device discovery is the first step where the server scans the network and sends out multicast Hello messages to find the connected PLCs. The second stage is the exchange of meta-information. The server receives from PLCs the information about process variables and data structures. The final step is auto-configuration where the OPC UA server automatically builds the information model within its address space and establishes the necessary mappings. After this process is done, the server is a master protocol, to constantly synchronize information with industrial devices. Overall OPC UA reduces deployment time and increases system flexibility. [15]

3) *Security Mechanisms and Provisioning Vulnerabilities*: Security is one of the major features of OPC UA. To protect industrial communication from cyber attacks, OPC UA provides mechanisms that ensure data authenticity, integrity and confidentiality. It supports three security modes including None, Sign and SignAndEncrypt. Secure communication

requires a handshake process in which devices exchange and verify X.509 certificates using local trust lists. [16]

Before secure communication can take place, devices must undergo a secure provisioning process which transfers them from a manufacturing state to an operational state. The current OPC UA standard uses a Global Discovery Server (GDS) for certificate management. However, the initial connection process relies on a Trust on First Use (TOFU) approach. The absence of initial authentication creates opportunities for cyber attacks such as Man-in-the-Middle attacks, credential interception and forged certificate deployment. [16]

Due to the complexity of provisioning requirements, many OPC UA devices deployed on the Internet are configured with security weaknesses such as disabled security modes, inadequate access control and certificate reuse. To address these challenges, the emerging OPC 10000-21 standard proposes the use of asymmetric key pairs and IEEE 802.1AR Initial Device Identity certificates during manufacturing. This approach establishes a cryptographic trust anchor and provides secure identity management throughout the device lifecycle. These security improvements can strengthen trust and reduce vulnerabilities in OPC UA-based industrial systems. [16].

C. CoAP

The Constrained Application Protocol is a lightweight web transfer protocol that is specifically designed for devices with limited processing power, RAM and networks with low bandwidth. CoAPs core design fundamentals align heavily with IIoT nodes' capabilities [4].

CoAP uses REST architecture which is similar to HTTP, and uses request response methods like GET, PUT, POST, DELETE etc. CoAP is built on UDP to achieve its lightweight design. It has its own simple message layer to handle re-transmission of lost data packages, reducing complexity and keeping header size as small as 10-20 bytes [4] but it has been further reduced to 4 bytes on average nowadays [4].

Instead of resource-heavy polling through constant data pulling from devices, CoAP uses Push Model as an asynchronous observer pattern where a client can "observe" a resource and will push notification automatically only when the data changes value.

On its own, CoAP is already very useful, communicating between only CoAP nodes, but it maximizes its scope by interworking with HTTP. CoAP is highly inter-operable with HTTP and works seamlessly with traditional web [4]. Proxies and gateways can translate between CoAP and HTTP easily, making communication between web browsers and IIoT devices that use CoAP hassle free. It is possible to build intermediaries that speak CoAP on one site and HTTP on another, without having to encode specific application knowledge.

CoAP is designed to handle small payloads and can easily handle we can expect to get in IoT systems, such as temperature sensors, light switches, pressure sensors and proximity sensors. However, applications occasionally need to transfer larger payloads, for example, for firmware updates. HTTP can be utilized in those cases to slice up large CoAP data packaged

into multiple manageable data packets, ensuring they arrive in the right order and handled as such.

In the industrial IoT environment, the devices must be able to discover each other. CoAP has a process to ensure its nodes for discovering and advertising resource descriptions. To ensure resource discovery, CoAP servers are encouraged to provide a resource description available via URI /.well-known/core for resource discovery. Clients then access this description with a GET request on that URI. The same description could be advertised, or even posted to a description directory. The protocol is highly optimized for Class-1 devices that usually have around 10 KB of RAM and 100 KB of program space [4]. Such devices are constrained enough to support conventional protocols like HTTP, TLS, and XML, but powerful enough to be part of the Internet through CoAP in order to optimize battery usage which may last several years with a single battery charge.

IV. CASE STUDY(PRACTICAL ANALYSIS)

This case study presents a practical comparison between MQTT and OPC UA, two widely used communication protocols in IIoT environments although this paper discusses CoAP, MQTT and OPC UA. Both protocols were evaluated in an industrial communication environment using a Node-RED testbed with a simulated factory temperature sensor. The evaluation is based on latency, message overhead, architecture and security performance. Active cyber attack simulations are also performed under different security conditions to examine how each protocol responds to unauthorized data injection and security threats.

A. Simulation Methodology

In this simulation Node-RED, an open-source flow-based programming environment, was used to implement the system on a local Windows machine. The simulation was designed in such a way that both MQTT and OPC UA communication protocols operate at the same time using a single sensor source. This allows both protocols to receive the same data under identical conditions. Wireshark was also used during the simulation to capture and analyze network traffic generated by both communication pipelines.

1) *Testbed Components:* Several software components were used in this simulation. The MQTT broker was implemented with the package Aedes and configured to work on the port 8883 with TLS 1.2 security. The OPC UA server was implemented using the `node-red-contrib-opcua` package and configured with Basic256Sha256 security policy and SignAndEncrypt security mode. For real-time monitoring and visualization, the Node-RED dashboard was used. Certificate management was performed using OpenSSL where a dedicated Certificate Authority (CA) was created to sign the legitimate server certificate and verify trusted client certificates during communication.

2) *Sensor Simulation and Payload Design:* The sensor simulation starts from a single inject node which triggers every two seconds. A function node generates a random temperature

value between 75°C and 85°C and creates a payload in the format `temperature|timestamp|SECRET_TOKEN`. The timestamp records the exact time when the message is generated and the secret token acts as a shared verification key between trusted devices. The generated payload is then sent simultaneously to both MQTT and OPC UA communication paths. In this way both protocols receive the same data at the same time and can be compared fairly.

3) *MQTT Pipeline*: In the MQTT communication path, the generated payload is published to the topic `factory/temperature` through the TLS secured Aedes broker running on port 8883. The subscriber node receives the message and forwards it to the latency calculation function. This function records the receive time, separates the payload fields, verifies the secret token, and calculates the end-to-end latency using the difference between the receive timestamp and the embedded send timestamp. If the token is valid then the dashboard displays the message and latency value with a verified status. If the token is missing or incorrect, the system shows a notification that an attack has been blocked.

4) *OPC UA Pipeline*: The first one is the node identifier `ns=1;s=MachineTemp` formatted in the payload and the String datatype in the OPC UA communication path. The message is then sent to the OPC UA server using a write client. During the system startup, an initialization process creates the `MachineTemp` variable in the address space of the OPC UA server. A separate read client continuously retrieves the value from the server every ten seconds. The received data is processed and forwarded to the latency calculation function where the same token verification and latency measurement procedures used in MQTT are applied.

5) *Message Overhead Observation*: To examine communication overhead, Wireshark was used to capture network traffic during the operation of both protocols. MQTT packets were filtered using port 8883 while OPC UA packets were filtered using port 53880. Packet sizes and protocol framing information were recorded and compared. Through this process we can observe the message overhead introduced by each protocol under their respective security configurations.

6) *Security Attack Simulation*: In this simulation four different attack scenarios were implemented to examine how MQTT and OPC UA respond to unauthorized data injection attempts. For each protocol, two attack conditions were tested. The first condition used incorrect security credentials while the second condition used valid credentials but malicious data. This allows us to observe the security behavior of each protocol at different layers.

For MQTT, the first attack scenario was when a hacker client attempted to connect with a certificate that was not signed by the trusted Certificate Authority. When the connection request was made, the broker did mutual TLS verification and immediately rejected the client because (i) the certificate could not be trusted. TLS handshake failed, no data was sent and the subscriber received nothing. This shows how MQTT with correctly configured mutual TLS can prevent unauthorized users from reaching the broker.

In the second MQTT attack scenario, the hacker client used a valid CA-signed certificate and connected successfully to the broker. Later the client released a spoofed payload without the secret token. Because the certificate was trusted, the broker accepted the message and delivered it to the subscriber. The application-layer verification function, however, found the missing token and rejected the message. This shows that TLS protects the communication channel and authenticates the client, but it doesn't verify the integrity of the message content itself.

The first attack scenario on OPC UA was a hacker client that tried to connect with an incompatible security policy. The client used Basic192, but the server had Basic256Sha256. The server rejected the connection request due to this mismatch and no session was established. No data was written into the server address space, and the read client did not receive any spoofed value. This scenario shows that OPC UA applies security policy requirements before communication can be started.

In the second OPC UA attack scenario the hacker client used the same security settings as the legitimate client, i.e. the correct security policy and security mode. So the server accepted the connection and let the client write a spoofed value directly to the `MachineTemp` node. Later when the read client fetched this value, the application layer validation process noticed the missing secret token and blocked the message. This demonstrates that, even if an attacker gains access with valid credentials, application level verification further ensures that the system is protected against unauthorized data manipulation.

Each attack scenario was performed manually while the two protocols were running. During the process, dashboard outputs, Node-RED status indicators, and Wireshark packet captures were recorded. By completing these experiments we can examine how MQTT and OPC UA enforce security at different layers and how cyber-physical communication systems can detect and respond to unauthorized activities.

B. Results and Evaluation

1) *Latency*: In this simulation we measured the latency performance of MQTT and OPC UA under normal operating conditions. The results show that MQTT delivered messages with latency values between 4 ms and 7 ms, while OPC UA delivered messages between 8 ms and 12 ms. The lowest recorded values were 4 ms for MQTT and 8 ms for OPC UA when both protocols received the same sensor value of 81.90°C, confirming that both pipelines operated with identical input data. From the results we can see that MQTT provides faster message delivery because messages are forwarded directly through the broker, whereas OPC UA requires separate write and read operations which introduce additional delay. Therefore, MQTT is more suitable for applications requiring fast and continuous sensor data transmission.

2) *Message Overhead*: To examine message overhead, network traffic was captured using Wireshark during the operation of both protocols. The results show that MQTT communication on port 8883 generated uniform packet sizes of 132 bytes

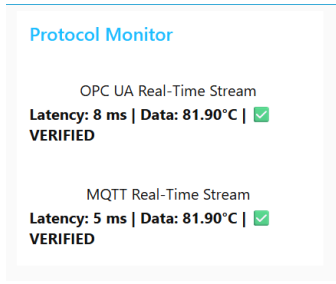


Fig. 2. Protocol monitor dashboard displaying real-time latency and data verification for OPC UA and MQTT streams.

under TLS encryption, with the payload fully encrypted. In comparison, OPC UA communication on port 53880 generated packet sizes ranging from 44 bytes to 236 bytes depending on the operation being performed. These packets contained additional information such as session metadata, node identifiers, and datatype information together with the payload. From these observations we can see that MQTT produces a smaller and more consistent message footprint, while OPC UA generates larger and more variable packet sizes due to its richer communication structure. This makes MQTT more suitable for bandwidth-limited and high-frequency communication environments.

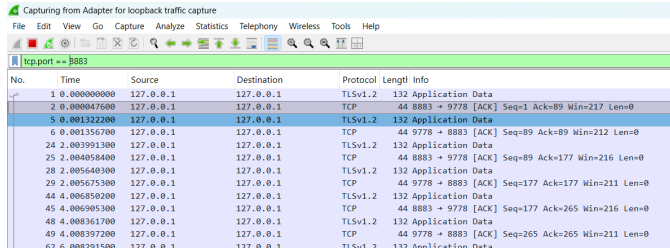


Fig. 3. Wireshark packet capture of MQTT traffic over port 8883, showing uniform 132-byte encrypted TLSv1.2 payloads.

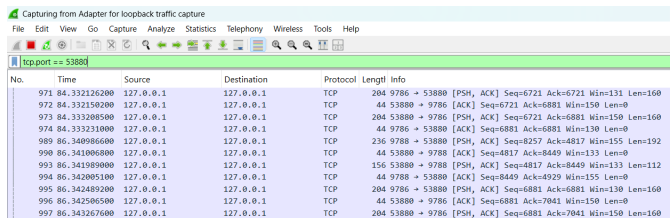


Fig. 4. Wireshark packet capture of OPC UA traffic over port 53880, demonstrating variable packet lengths associated with its communication structure.

C. Architectural Design

The implementation of MQTT and OPC UA required different system architectures. The MQTT communication path consisted of three main components: the broker, publisher, and subscriber, reflecting its lightweight publish-subscribe model. In contrast, the OPC UA communication path required additional components including the server, address space

configuration, write client, read client, and structured node management. Although this increases complexity, it provides better support for structured data representation and semantic information. From this comparison we can see that MQTT is simpler and easier to deploy, while OPC UA offers more advanced capabilities for industrial systems requiring detailed data organization.

1) *Security Performance:* In this study four attack scenarios were performed to evaluate the security performance of both protocols, with two scenarios implemented for MQTT and two for OPC UA.

For MQTT, the first attack scenario used a certificate that was not signed by the trusted Certificate Authority. The TLS handshake was rejected and the connection was blocked before any message entered the system. In the second scenario, the hacker used a valid CA-signed certificate and successfully connected to the broker. However, the spoofed payload did not contain the correct secret token and was detected by the application-layer verification mechanism. This shows that TLS verifies the identity of the client, but additional application-level checks are required to verify message integrity.

For OPC UA, the first attack scenario involved a hacker using an incorrect security policy. Since the policy did not match the server configuration, the session establishment failed and no data exchange occurred. In the second scenario, the hacker used the same security configuration as the legitimate client and successfully established a session. The spoofed value was written into the server address space, but the missing secret token was detected by the application-layer verification process and the attack was blocked. This shows that OPC UA enforces security policy verification before communication begins, but application-level validation is still necessary for detecting manipulated payloads.

From all attack scenarios we can see that both MQTT and OPC UA successfully blocked unauthorized access when incorrect credentials or security settings were used. When valid credentials were presented, both protocols relied on the application-layer token verification mechanism to detect malicious payloads. The results show that transport-layer security alone is not sufficient to guarantee data integrity. Additional payload validation at the application layer plays an important role in protecting the system against spoofed or manipulated data. By completing these experiments we can see that both protocols provide strong security mechanisms, but application-layer verification remains an essential component of secure cyber-physical systems.

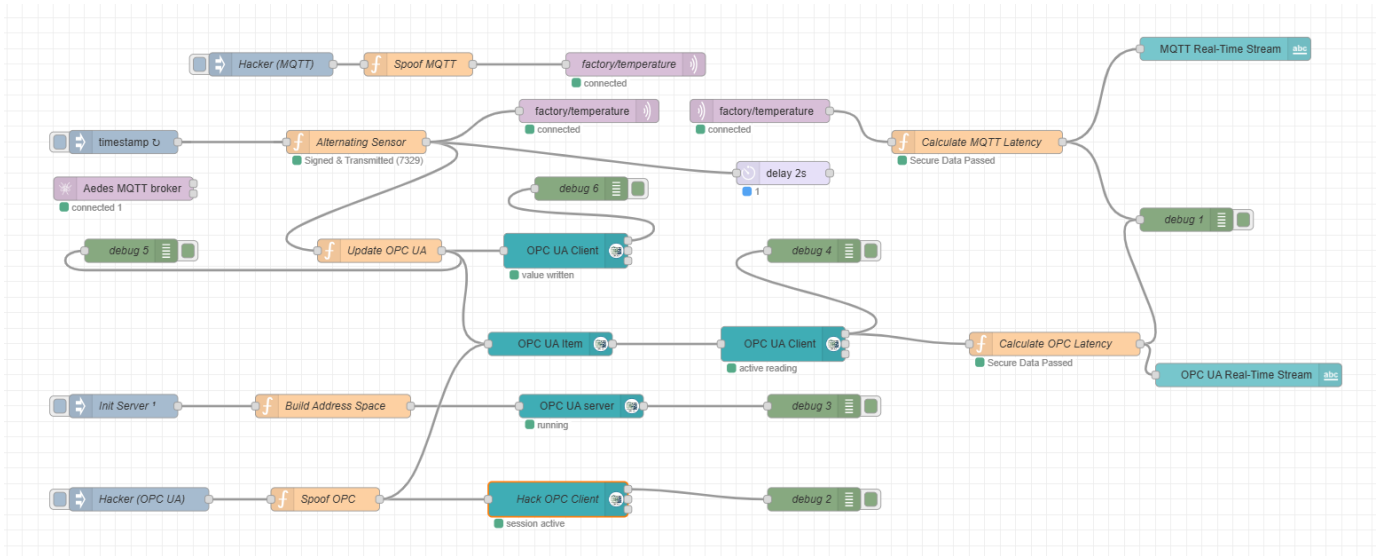


Fig. 5. Node-RED implementation flow illustrating the architectural design of the MQTT and OPC UA communication pipelines, alongside testing and attack simulation nodes.

V. DATA ACQUISITION, MONITORING, AND CLOUD INTEGRATION

In modern IIoT architectures, communication protocols are not only used for local data exchange but also for connecting industrial systems with supervisory systems and cloud platforms. The movement of data from physical devices to cloud-based analytics creates an important communication process where MQTT, OPC UA, and CoAP play different roles across the device-to-edge and edge-to-cloud spectrum [21].

For local data acquisition and monitoring, OPC UA is widely used to connect field devices horizontally with SCADA and MES systems. It provides a standardized way of handling machine data by using complex information modeling that adds semantic meta-information to the actual data before it is displayed or processed [22]. On the other hand, CoAP is suitable for resource-constrained and battery-operated sensors because its lightweight design uses fewer network resources (such as RAM, ROM, and power) while transmitting data to monitoring systems over lossy networks [21].

As industrial applications continue to adopt data analytics and cloud computing, cloud integration becomes an important requirement. MQTT is commonly used to transfer data from edge gateways to cloud platforms because of its decoupled publish-subscribe architecture and low bandwidth requirements, making it suitable for sending telemetry data over fragile or unreliable network connections [21]. While the traditional OPC UA client-server model is mainly used for in-plant communication inside factory networks, recent developments such as the OPC UA PubSub extension combined with MQTT as a broker-based middleware are helping to connect factory systems directly with cloud platforms [23]. In this process, secure, semantically rich, and contextualized industrial data can be effectively aggregated and transferred from the factory floor to cloud-based applications. Overall

these protocols support efficient communication, scalability, and data integration in modern IIoT systems.

VI. SECURITY AND RELIABILITY CONSIDERATIONS

One of the most important aspects to consider before implementing a certain network for industry is how secure the network is and what security protocols are available in the protocol.

A. Current Challenges in Security

The actual adoption of TLS is very low; recent scans show that only around 6.5% of IIoT deployments use it and even when it is used, it's often done badly. Outdated versions are still deployed like TLS 1.0 or 1.1, along with insecure cipher suites and even self-signed certificates [18].

Another big issue is resource constraints versus cryptographic overhead. Secure methods like Public Key Cryptography or DNSSEC use up too much computing power and drain batteries on edge devices. These devices need to work in real time, but strong security can really hold things back. Access control also falls flat. Too many deployments skip proper application-layer controls. For example, many MQTT and AMQP brokers either stay wide open to the Internet or depend on default login info, making them easy targets for bad actors [20].

On top of that, hackers can create elaborate schemes that fit right in with normal network activity, using valid IPs and protocols. Attacks like Stuxnet or Industroyer sneak past traditional NIDS because they follow standard procedure. Lastly, secure systems aren't safe from everything. While OPC UA does well inside tightly controlled manufacturing networks, accessing clients from outside means opening up internal firewall ports. This creates risk, even in otherwise solid systems [17].

TABLE II
PROTOCOL SECURITY AND RELIABILITY COMPARISON

Protocol	Transport Layer	Principal Security Philosophy & Architecture	Reliability & Performance
MQTT	TCP	Relies heavily on standard TLS/SSL for strong data encryption and integrity, but lacks in-built application-layer cryptography. Access control is primarily handled by the broker.	Exhibits high variability and packet loss under heavy parallel loads due to a lack of native queueing.
CoAP	UDP	Designed for resource-constrained (Class I) devices and lossy networks where traditional TLS is impractical. Natively supports DTLS and OSCORE for end-to-end application-level encryption.	Achieves the lowest Time-to-Completion (TTC) for highly frequent messages, though setup mechanisms can cause latency spikes.
OPC UA	TCP/UDP	Features a deeply integrated, multi-layered security structure including X.509 authentication, AAA session security, and transport-level data signing.	Provides the highest stability and lowest sensitivity to message frequency variations, but imposes a higher resource burden and baseline latency.

B. Current Challenges in Reliability

Beyond just security, industrial automation needs low latency, between 10 and 100 milliseconds, and super high reliability, up to 99.99%. Co-existence issues, interference, and handling unexpected disruptions hinder these requirements. Deploying lots of wireless IIoT devices in already crowded spectrum bands, like ISM, causes major signal interference. Plus, many commercial edge gadgets can't handle complex spectrum-sensing algorithms or Forward Error Correction optimized for those short IIoT packets, typically 50 to 70 bytes long. These networks have to deal with sudden environmental changes or link failures too. Current systems relying on centralized scheduling usually can't keep bounded response times during these disturbances, as they often lack the needed scalability [20].

C. Future Outlook and Mitigation Strategies

To address the ever-increasing security and reliability challenges, the industry is moving toward several advanced solutions. A few of the security systems being introduced in the industry today are discussed as a future path that is possible.

- **Process-aware intrusion detection systems:** In order to address advanced attacks, security frameworks of the future will use distributed tracing and process mining to model IoT processes as expected workflows. Solutions such as MISSION can trace MQTT communication patterns and model them into a Petri-net. By understanding the context and exact message flows (i.e., by correlating Publish to preceding Subscribe messages), an anomaly-based NIDS system can then detect anomalies at a process level without impacting performance [17].
- **Automated certificate management and safe templating:** In order to address the common problem of incorrect certificate configuration and weak ciphers, solutions of the future should adopt secure-by-design deployment templates and actively maintain up-to-date guidelines for certificate management. This will ensure keeping cryptographic primitives used by industrial equipment up to date [18].
- **Hybrid Protocol Solutions:** No existing protocol will suffice for all cases. Instead, future security protocols should capitalize on the inherent strength of each protocol and combine them. For instance, OPC UA should be used

for its strong SOA and complicated information models to enable secure horizontal communication in plants, while other lightweight protocols such as MQTT and CoAP could be used for edge computing to the cloud [22].

- **Decentralized and Next Generation Networking:** With the use of decentralized resource management techniques and new networking technologies, including Time Sensitive Networks (TSN) along with OPC UA Pub Sub enhancements, 5G URLLC, and Information Centric Networking concepts such as NDN, latency will be drastically reduced, while packet loss will improve and security will be inherent in the data transport process [22].

VII. CONCLUSION

The use of IIoT protocols has an important role in modern Industry 4.0 factories connecting the physical machines with the computer networks in a secure manner. Data transmission between devices, control of network traffic and limitation of unauthorized access to industrial systems are carried out using various communication protocols. This paper has presented a comparative analysis of different IIoT protocols including MQTT, OPC UA and CoAP based on communication speed, message size and resistance against cyber attacks. We considered the advantages and limitations of each protocol in industrial environments in doing so. Our results show that MQTT is suitable for continuous sensor data exchange with fast communication and light weight data transmission. OPC UA has better data organization and strong built-in security features, which make it suitable for complex industrial networks, while CoAP is efficient for communication with simple and battery powered devices. This work allows us to conclude that maintaining secure and efficient industrial systems requires the selection of the appropriate protocol according to the needs of the application and the constant verification of the transmitted data.

ACKNOWLEDGMENT

Muhammad Aamir Ejaz Ejaz was responsible for IIoT Communication Requirements and MQTT. **Md Ruhul Kuddus Saleh** was responsible for the OPC UA and the simulation work. **Ikramul Hassan Sazid** was responsible for the CoAP, Security and reliability section.

The authors would like to express their sincere gratitude to **Prof. Dr. Ali Hayek** for his continuous guidance, constructive

feedback, and invaluable support throughout the research and writing process. His insightful comments and deep knowledge of IIoT communication protocols helped shape the direction and depth of this paper. This work would not have been possible without his dedicated mentorship

Tools we have used:

- Grammarly - Used for checking spelling and correcting grammatical errors.
- Quillbot - Used for Paraphrasing.
- Node-Red - Used for simulation work.
- Python-Script - Used for plotting graph.
- WireShark - Used for simulation work
- LLM - Used to assist with table formating and idea generation.

REFERENCES

- [1] K. Tange, M. De Donno, X. Fafoutis, and N. Dragoni, "A Systematic Survey of Industrial Internet of Things Security: Requirements and Fog Computing Opportunities," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2489-2520, Fourth Quarter 2020.
- [2] M. Reisig and J. Lenz, "Application and Evaluation of Communication Protocols in Industry 4.0: Use Case Study with End-to-End Implementation from Data Acquisition to Data Visualization," *Procedia CIRP*, vol. 139, pp. 219-224, 2026.
- [3] F. Ercan, M. Bega, and B. Kuhlentötter, "A systematic literature review of communications standards in discrete manufacturing," *5th Conference on Production Systems and Logistics*, Chair of Production Systems/Ruhr-University Bochum, Bochum, Germany.
- [4] D. Silva, L. I. Carvalho, J. Soares, and R. C. Sofia, "A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA," *Applied Sciences*, vol. 11, no. 11, p. 4879, May 2021.
- [5] I. Behnke and H. Austad, "Real-Time Performance of Industrial IoT Communication Technologies: A Review," *IEEE Access*, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10317890>
- [6] F. H. Cabrini, F. Valiante Filho, P. Rito, A. Barros Filho, S. Sargento, A. Venâncio Neto, and S. T. Kofuji, "Enabling the Industrial Internet of Things to Cloud Continuum in a Real City Environment," *Sensors*, vol. 21, no. 22, p. 7707, Nov. 2021. DOI: 10.3390/s21227707
- [7] G. Bravos *et al.*, "Cybersecurity for industrial internet of things: Architecture, models and lessons learned," *IEEE Access*, vol. 10, pp. 124747-124765, 2022, doi:10.1109/ACCESS.2022.3225074.
- [8] "Strategies for Assuring Low Latency, Scalability and Interoperability in Edge Computing and TSN Networks for Critical IIoT Services," *IEEE Xplore*, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10105239>
- [9] K. Parsa and M. Naderpour, "Product Lifecycle and RAMS Requirement at Industry 4.0," *Product Lifecycle Management*, p. 21, 2025.
- [10] "5G Support for Industrial IoT Applications — Challenges, Solutions, and Research Gaps," *PMC / NCBI*, 2020. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7038716/>
- [11] B. Mishra and A. Kertész, "The Use of MQTT in M2M and IoT Systems: A Survey," *IEEE Access*, vol. 8, pp. 201071-201086, 2020, doi: 10.1109/ACCESS.2020.3035849.
- [12] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>
- [13] C. O. Del Socorro, A. A. D. Susada, M. D. Tapuli, and J. A. P. Esparcia, "A Qualitative Comparative Analysis of MQTT, HTTP/HTTPS, CoAP, AMQP, and OPC UA for Internet of Things Applications," *International Journal of Engineering Research & Technology (IJERT)*, vol. 15, no. 5, May 2026.
- [14] E. Tapia, L. Sastoque-Pinilla, U. Lopez-Novoa, I. Bediaga, and N. López de Lacalle, "Assessing Industrial Communication Protocols to Bridge the Gap between Machine Tools and Software Monitoring," *Sensors*, vol. 23, no. 12, p. 5694, Jun. 2023.
- [15] J. M. Gutierrez-Guerrero and J. A. Holgado-Terriza, "Automatic Configuration of OPC UA for Industrial Internet of Things Environments," *Electronics*, vol. 8, no. 6, p. 600, May 2019.
- [16] F. Kohnhäuser, D. Meier, F. Patzer, and S. Finster, "On the Security of IIoT Deployments: An Investigation of Secure Provisioning Solutions for OPC UA," *IEEE Access*, vol. 9, pp. 99299-99311, 2021.
- [17] P. Empl, F. Böhm, and G. Pernul, "Process-Aware Intrusion Detection in MQTT Networks," in *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy (CODASPY '24)*, Porto, Portugal, Jun. 2024.
- [18] M. Dahlmanns, J. Lohmöller, J. Pennekamp, J. Bodenhausen, K. Wehrle, and M. Henze, "Missed Opportunities: Measuring the Untapped TLS Support in the Industrial Internet of Things," in *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security (ASIA CCS '22)*, Nagasaki, Japan, May 2022.
- [19] M. Ladegourdie and J. Kua, "Performance Analysis of OPC UA for Industrial Interoperability towards Industry 4.0," *IoT*, vol. 3, pp. 507-525, Dec. 2022.
- [20] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, "Industrial Internet of Things: Challenges, Opportunities, and Directions," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 11, pp. 4724-4734, Nov. 2018.
- [21] D. Silva, L. I. Carvalho, J. Soares, and R. C. Sofia, "A performance analysis of Internet of Things networking protocols: Evaluating MQTT, CoAP, OPC UA," *Applied Sciences*, vol. 11, no. 11, p. 4879, 2021.
- [22] M. Ladegourdie and J. Kua, "Performance analysis of OPC UA for industrial interoperability towards Industry 4.0," *IoT*, vol. 3, no. 4, pp. 507-525, 2022.
- [23] H. Raddatz, E. Mahmoud, F. Hölzke, P. Danielis, D. Timmermann, and F. Golatowski, "Evaluation and extension of OPC UA publish/subscribe MQTT binding," in *2020 IEEE Conference on Industrial Cyberphysical Systems (ICPS)*. IEEE, 2020, pp. 543-548.